

COHORT_OF_SONGS_Improved

May 15, 2026

MACHINE LEARNING Course End Project Creating Cohorts of Songs

15-05-2026

```
[1]: import pandas as pd
      from sklearn.preprocessing import PowerTransformer
```

Import dataset

```
[2]: df=pd.read_csv('rolling_stones_spotify.csv')
```

Drop unneeded columns

```
[3]: df=pd.read_csv('rolling_stones_spotify.csv')
      try:
          df.drop('Unnamed: 0', axis=1, inplace=True)
      except:
          pass
      og_df = df.copy()
```

```
[4]: list_to_drop = ['name', 'album', 'track_number', 'id', 'uri']
      df.drop(list_to_drop, axis=1, inplace=True)
```

Initial data inspection and data cleaning: Examine the data initially to identify duplicates, missing values, irrelevant entries, or outliers. Check for any instances of erroneous entries and rectify them as needed

```
[5]: def do_binning(df):
      df = df.copy()
      # Assuming your DataFrame is named 'df' and has a 'release_date' column
      # First, ensure 'release_date' is in datetime format
      df['release_date'] = pd.to_datetime(df['release_date'], errors='coerce')
      # Extract the year
      df['year'] = df['release_date'].dt.year
      # Generate bins and labels dynamically based on min and max year
      min_year = df['year'].min()
      max_year = df['year'].max()
      # Calculate start and end decades
      start_decade = (min_year // 10) * 10
      end_decade = ((max_year // 10) + 1) * 10
```

```

# Generate bins using range
bins = list(range(start_decade, end_decade + 10, 10))
# Generate labels
labels = [f"{decade}s" for decade in range(start_decade, end_decade, 10)]
# Bin the years into decades
df['decade'] = pd.cut(df['year'], bins=bins, labels=labels, right=False)
# Optional: Handle any NaN values or years outside the bins
df['decade'] = df['decade'].cat.add_categories(['Unknown']).
↳ fillna('Unknown')
# Now df has a new 'decade' column with the binned values
df.drop(['release_date', 'year'], axis=1, inplace=True)
return df

def detect_outliers(df, column, factor=1.5):
    """
    Detect outliers in a skewed distribution using the IQR method.
    """
    values = df[column].dropna()
    q1 = values.quantile(0.25)
    q3 = values.quantile(0.75)
    iqr = q3 - q1
    lower = q1 - factor * iqr
    upper = q3 + factor * iqr
    outliers = values[(values < lower) | (values > upper)]
    return outliers.index.tolist(), outliers

```

Refine the data for further processing based on findings

```

[6]: ndf = do_binning(df)

numerical_cols = ndf.select_dtypes(include=['float64', 'int64']).columns #↳
↳ Adjust based on your data

scaler = PowerTransformer()
scaler.fit(ndf[numerical_cols])

ndf[numerical_cols] = scaler.transform(ndf[numerical_cols])

ndf_encoded = pd.get_dummies(ndf, columns=['decade'], prefix='decade',
↳ dtype=int)

try:
    ndf_encoded.drop('decade_Unknown', axis=1, inplace=True)
except:
    pass

```

```
[28]: print(ndf_encoded.duplicated().sum())
duplicates = ndf_encoded[ndf_encoded.duplicated(keep=False)].
↳ sort_values(by=list(ndf_encoded.columns))
```

0

Perform exploratory data analysis and feature engineering

```
[8]: duplicates
```

```
[8]:
```

	acousticness	danceability	energy	instrumentalness	liveness	\
913	-1.293011	-0.174281	1.245848	-0.472237	-0.167193	
935	-1.293011	-0.174281	1.245848	-0.472237	-0.167193	
918	-0.890525	1.701000	0.276070	-0.366367	-0.160912	
940	-0.890525	1.701000	0.276070	-0.366367	-0.160912	
917	0.233631	0.892544	-0.428987	-0.814894	0.727026	
939	0.233631	0.892544	-0.428987	-0.814894	0.727026	
907	0.993687	-0.589461	1.451741	1.802783	0.494274	
929	0.993687	-0.589461	1.451741	1.802783	0.494274	
906	1.241576	0.074266	0.871709	1.653476	-0.442607	
928	1.241576	0.074266	0.871709	1.653476	-0.442607	
924	1.720455	2.399059	-1.110012	0.499042	-1.135447	
946	1.720455	2.399059	-1.110012	0.499042	-1.135447	

	loudness	speechiness	tempo	valence	popularity	duration_ms	\
913	0.864315	1.473814	-0.685914	-0.955287	-0.178461	-0.216256	
935	0.864315	1.473814	-0.685914	-0.955287	-0.178461	-0.216256	
918	0.238802	-0.733944	0.108633	1.165374	-0.351627	0.039971	
940	0.238802	-0.733944	0.108633	1.165374	-0.351627	0.039971	
917	0.528741	-1.271257	-2.043961	-0.220193	-0.013197	0.279036	
939	0.528741	-1.271257	-2.043961	-0.220193	-0.013197	0.279036	
907	2.621323	1.860636	1.217522	-0.120844	-0.263975	-0.581252	
929	2.621323	1.860636	1.217522	-0.120844	-0.263975	-0.581252	
906	1.818203	0.919413	-2.074497	-0.369940	0.145267	0.340061	
928	1.818203	0.919413	-2.074497	-0.369940	0.145267	0.340061	
924	-0.486298	-0.655157	-0.290589	-1.068028	-0.534143	-2.122311	
946	-0.486298	-0.655157	-0.290589	-1.068028	-0.534143	-2.122311	

	decade_1960s	decade_1970s	decade_1980s	decade_1990s	decade_2000s	\
913	0	1	0	0	0	
935	0	1	0	0	0	
918	0	1	0	0	0	
940	0	1	0	0	0	
917	0	1	0	0	0	
939	0	1	0	0	0	
907	0	1	0	0	0	
929	0	1	0	0	0	
906	0	1	0	0	0	

928	0	1	0	0	0
924	0	1	0	0	0
946	0	1	0	0	0

	decade_2010s	decade_2020s
913	0	0
935	0	0
918	0	0
940	0	0
917	0	0
939	0	0
907	0	0
929	0	0
906	0	0
928	0	0
924	0	0
946	0	0

```
[27]: ndf_encoded.drop_duplicates(inplace=True)
```

Detecting outliers for each column

```
[10]: num_list = ndf_encoded.select_dtypes(include=['float64', 'int64'])
outlier_ind_list = []

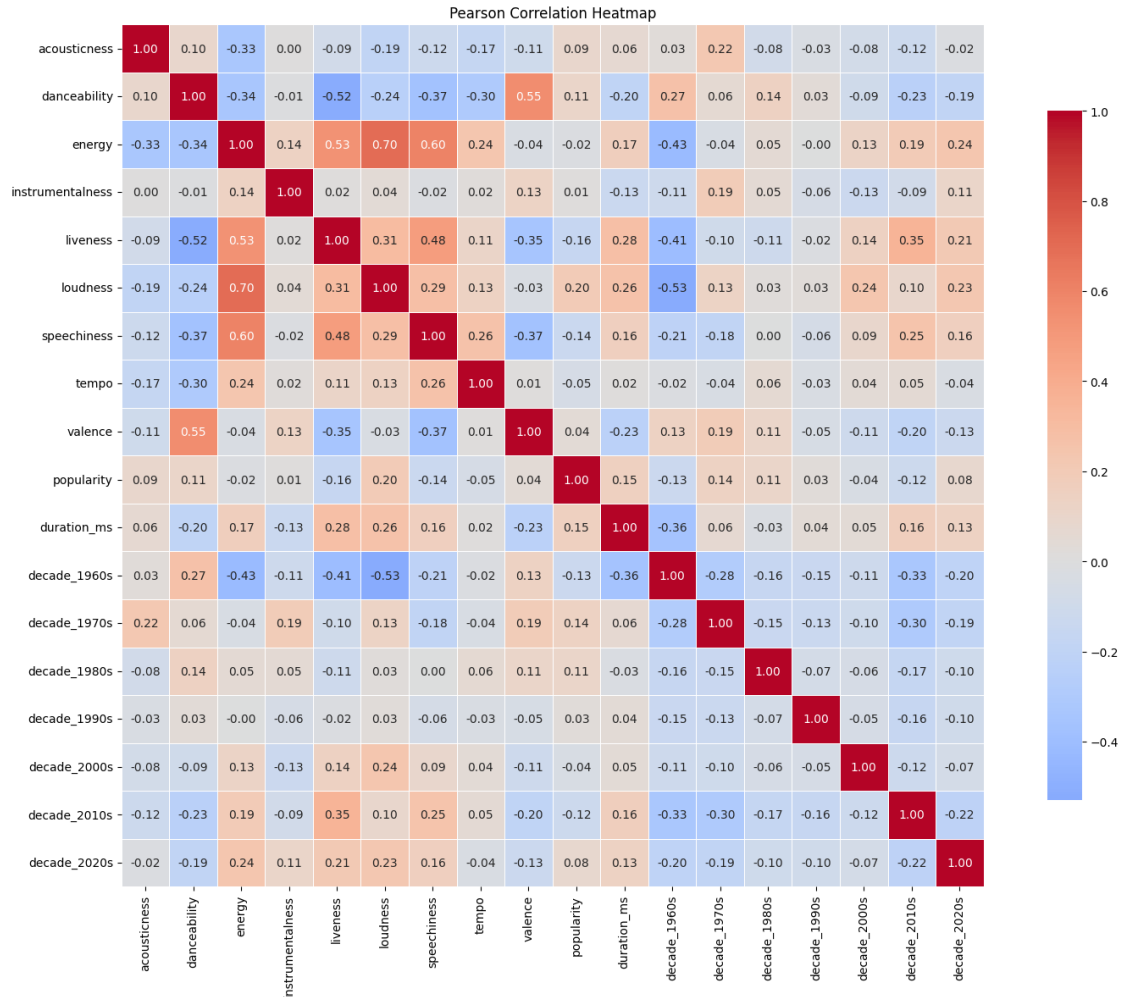
for col in num_list.columns:
    outlier_indices, outliers = detect_outliers(ndf_encoded, col)
    outlier_ind_list.append(outlier_indices)
    # print(f"Outliers in column '{col}':")
    # display(outliers)
```

Examine the relationship between a song's popularity and various factors, exploring how this correlation has evolved

```
[11]: #Correlation Heatmap
import seaborn as sns
import matplotlib.pyplot as plt

def heatmap(corr_matrix):
    plt.figure(figsize=(16, 12))
    sns.heatmap(corr_matrix, annot=True, fmt='.2f', cmap='coolwarm', center=0,
                square=True, linewidths=0.5, cbar_kws={"shrink": 0.8})
    plt.title('Pearson Correlation Heatmap')
    plt.tight_layout()
    plt.show()
```

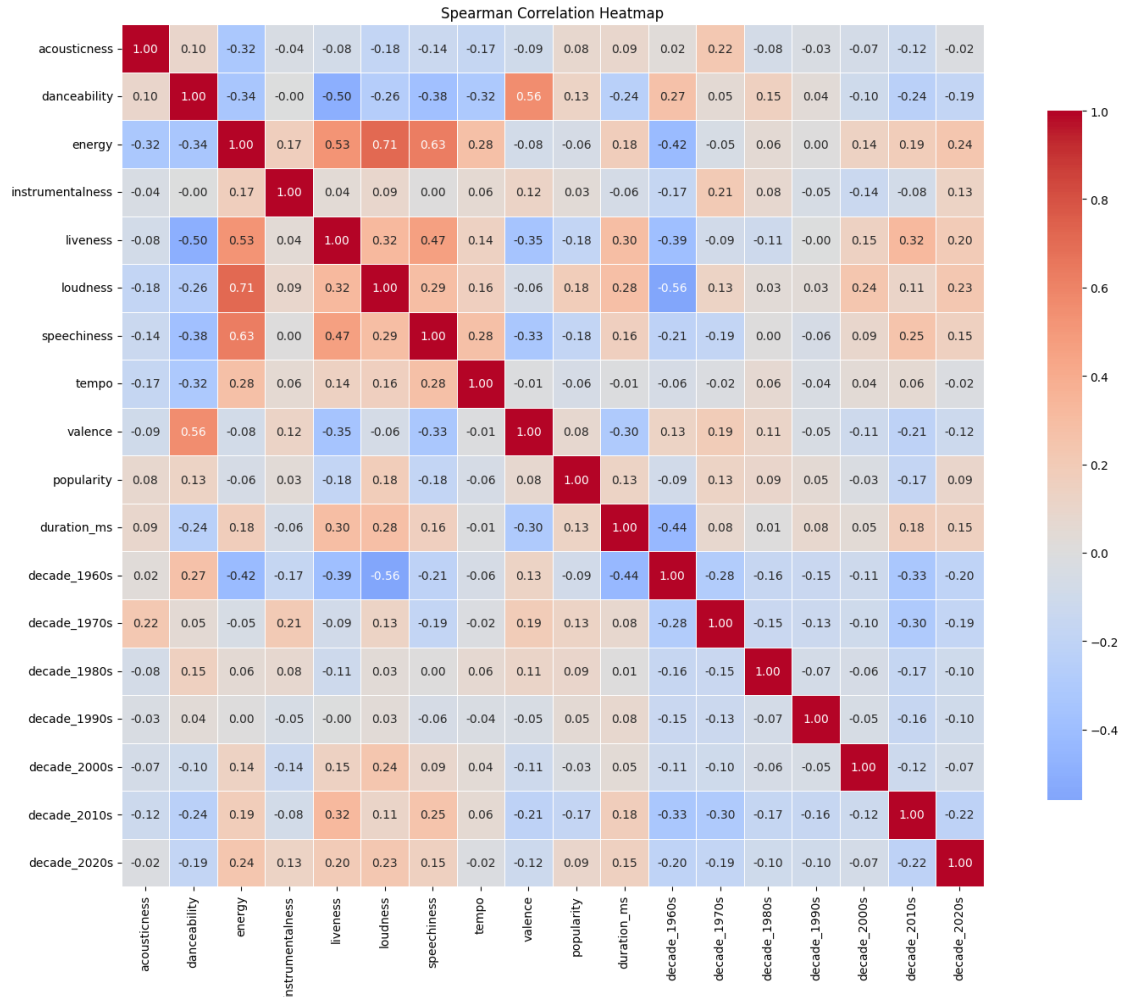
```
[12]: heatmap(ndf_encoded.corr(method='pearson'))
```



```
[13]: #Correlation Heatmap
import seaborn as sns
import matplotlib.pyplot as plt

def heatmap(corr_matrix):
    plt.figure(figsize=(16, 12))
    sns.heatmap(corr_matrix, annot=True, fmt='.2f', cmap='coolwarm', center=0,
square=True, linewidths=0.5, cbar_kws={"shrink": 0.8})
    plt.title('Spearman Correlation Heatmap')
    plt.tight_layout()
    plt.show()

[14]: heatmap(ndf_encoded.corr(method='spearman'))
```



Determining if PCA is needed

```
[15]: len(df.columns)
```

```
[15]: 12
```

```
[16]: from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
import numpy as np

# Apply PCA to reduce dimensionality
# Specify the number of components to keep (e.g., 2 for visualization or 0.95
#   ↳ for 95% variance)
pca = PCA(n_components=0.95) # Keep 95% of variance

# Fit and transform the data
```

```

pca_data = pca.fit_transform(ndf_encoded)

# Get explained variance
explained_variance = pca.explained_variance_ratio_
cumulative_variance = np.cumsum(explained_variance)

print(f"Number of components: {pca.n_components_}")
print(f"Explained variance ratio: {explained_variance}")
print(f"Cumulative explained variance: {cumulative_variance[-1]:.4f}")

# Plot explained variance
fig, ax = plt.subplots(1, 2, figsize=(14, 5))

# Scree plot
ax[0].plot(range(1, len(explained_variance) + 1), explained_variance, 'bo-')
ax[0].set_xlabel('Principal Component')
ax[0].set_ylabel('Explained Variance Ratio')
ax[0].set_title('Scree Plot')
ax[0].grid()

# Cumulative variance plot
ax[1].plot(range(1, len(cumulative_variance) + 1), cumulative_variance, 'ro-')
ax[1].axhline(y=0.95, color='g', linestyle='--', label='95% Variance')
ax[1].set_xlabel('Number of Components')
ax[1].set_ylabel('Cumulative Explained Variance')
ax[1].set_title('Cumulative Explained Variance')
ax[1].legend()
ax[1].grid()

plt.tight_layout()
plt.show()

# Create DataFrame with PCA results
pca_df = pd.DataFrame(
    pca_data,
    columns=[f'PC{i+1}' for i in range(pca.n_components_)]
)

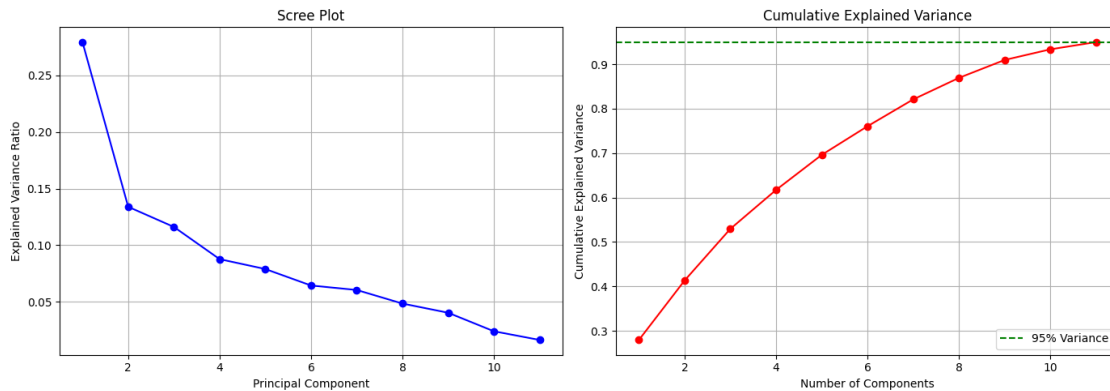
print("\nPCA transformed data shape:", pca_df.shape)
print(pca_df.head())

```

```

Number of components: 11
Explained variance ratio: [0.27946645 0.13394453 0.11614442 0.08769076
0.07895915 0.06440172
0.06044519 0.04845114 0.04032815 0.02396683 0.01630507]
Cumulative explained variance: 0.9501

```



PCA transformed data shape: (1604, 11)

	PC1	PC2	PC3	PC4	PC5	PC6	PC7	\
0	0.972452	0.002661	-2.068578	2.274469	0.000099	-2.663399	-2.420008	
1	2.124609	-0.337625	0.892882	1.749335	0.574837	-0.327587	-0.617792	
2	2.267020	-0.151091	0.827323	1.998422	0.404821	-0.143938	-0.861817	
3	2.663771	-1.504119	1.073773	0.189453	0.345277	0.385779	-1.715984	
4	2.544506	-1.097434	1.018420	0.698951	0.435036	-0.212677	-0.770431	

	PC8	PC9	PC10	PC11
0	1.137741	1.390103	0.955404	-0.756365
1	-0.130120	0.423207	0.212915	-0.612070
2	0.627241	0.305058	0.306773	-0.579134
3	0.485363	0.201286	0.064264	-0.734505
4	-0.014044	0.303340	0.043361	-0.715692

Observation: Number of components reduced to 11 => There is no need to use PCA

Utilize suitable visualizations to identify the two albums that should be recommended to anyone based on the number of popular songs in each album

Conduct exploratory data analysis to delve into various features of songs, aiming to identify patterns

Provide insights on the significance of dimensionality reduction techniques. Share your ideas and elucidate your observations

Examine the relationship between a song's popularity and various factors, exploring how this correlation has evolved

```
[17]: # Define what "high popularity" means
threshold = og_df['popularity'].quantile(0.75) # Top 25% as high popularity

# Count high-popularity songs per album
high_pop_counts = (
    og_df[og_df['popularity'] >= threshold]
```

```

        .groupby('album')['name']
        .count()
        .sort_values(ascending=False)
    )

display(high_pop_counts.head(10))

# Album with the most high-popularity songs
top_album = high_pop_counts[:2]
top_count = high_pop_counts[:2]
# print(f"\nAlbum with most high-popularity songs: {top_album} ({top_count} songs)")

# Optional: list the song names for that album
for talbum in top_album.items():
    top_album_songs = og_df[
        (og_df['album'] == talbum[0]) &
        (og_df['popularity'] >= threshold)
    ][['name', 'popularity']].sort_values(by='popularity', ascending=False)
    print(talbum)
    display(top_album_songs)

```

```

album
Honk (Deluxe)                18
Exile On Main Street (2010 Re-Mastered)  18
Exile On Main Street (Deluxe Version)    18
Licked Live In NYC           17
Live At The El Mocambo       14
Voodoo Lounge (Remastered 2009)  14
Bridges To Babylon (Remastered)  12
Out Of Our Heads             12
Blue & Lonesome              12
The Rolling Stones           12
Name: name, dtype: int64

```

```
('Honk (Deluxe)', 18)
```

	name	popularity
235	Doom And Gloom	49
240	Miss You	46
242	Don't Stop	44
234	Start Me Up	42
237	Beast Of Burden	42
233	Living In A Ghost Town	41
243	Tumbling Dice	41
254	Streets Of Love	38
274	Wild Horses - Live At London Stadium	35
238	Wild Horses - 2009 Mix	32

264	Happy	32
245	Rain Fall Down	28
250	Emotional Rescue	28
249	It's Only Rock'n'Roll (But I Like It)	28
253	Dancing With Mr. D	28
239	Angie	27
251	Waiting On A Friend	27
255	Fool To Cry	27

('Exile On Main Street (2010 Re-Mastered)', 18)

	name	popularity
1043	Tumbling Dice	63
1044	Sweet Virginia	56
1055	Shine A Light	53
1039	Rocks Off	52
1040	Rip This Joint	49
1041	Shake Your Hips	43
1050	Ventilator Blues	43
1053	All Down The Line	42
1052	Let It Loose	42
1047	Loving Cup	42
1045	Torn And Frayed	41
1048	Happy	41
1046	Sweet Black Angel	40
1042	Casino Boogie	40
1054	Stop Breaking Down	38
1049	Turd On The Run	37
1056	Soul Survivor	37
1051	I Just Want To See His Face	36

Perform cluster analysis

- Identify the right number of clusters
- Use appropriate clustering algorithms
- Define each cluster based on the features

Observation:

In order to use **KMeans** to perform cluster analysis, we will have to drop some values, because we are using numerical as well as categorical data. KMeans only works properly with numeric data, while **K-Prototypes** works with mixed numeric + categorical data.

If we switch from **KMeans** to **K-Prototypes**, we should generally use the dataframe before one-hot encoding.

For K-Prototypes, the “elbow” is based on cost, not KMeans-style WCSS.

```
[18]: # Drop columns whose names start with "decade"
```

```
ndf_encoded_nums = ndf_encoded.drop(columns=ndf_encoded.
↳filter(regex=r"~decade").columns)
```

```
[19]: #After dropping the columns beginning with "decade" and categorical variables
↳are dropped
ndf_encoded_nums
```

```
[19]:
```

	acousticness	danceability	energy	instrumentalness	liveness	\
0	-0.724013	-0.003342	1.409775	1.808126	1.204910	
1	0.993687	-1.010728	1.126955	1.134892	1.262425	
2	0.928436	-0.559889	1.166206	1.525095	1.252591	
3	1.337407	-0.686038	1.327030	-0.813353	1.129816	
4	0.876597	-1.187822	1.166206	-0.042070	1.272226	
...	...	...	...	...	...	
1605	-0.218553	0.017869	0.817229	-0.713819	-0.358183	
1606	-0.917981	0.318199	-0.731846	-0.815121	0.225701	
1607	0.777634	2.130132	-0.357896	-0.815149	-1.347873	
1608	0.118175	1.575863	-1.374211	-0.813971	-0.931695	
1609	0.819393	1.744527	0.835300	0.094031	-1.218275	

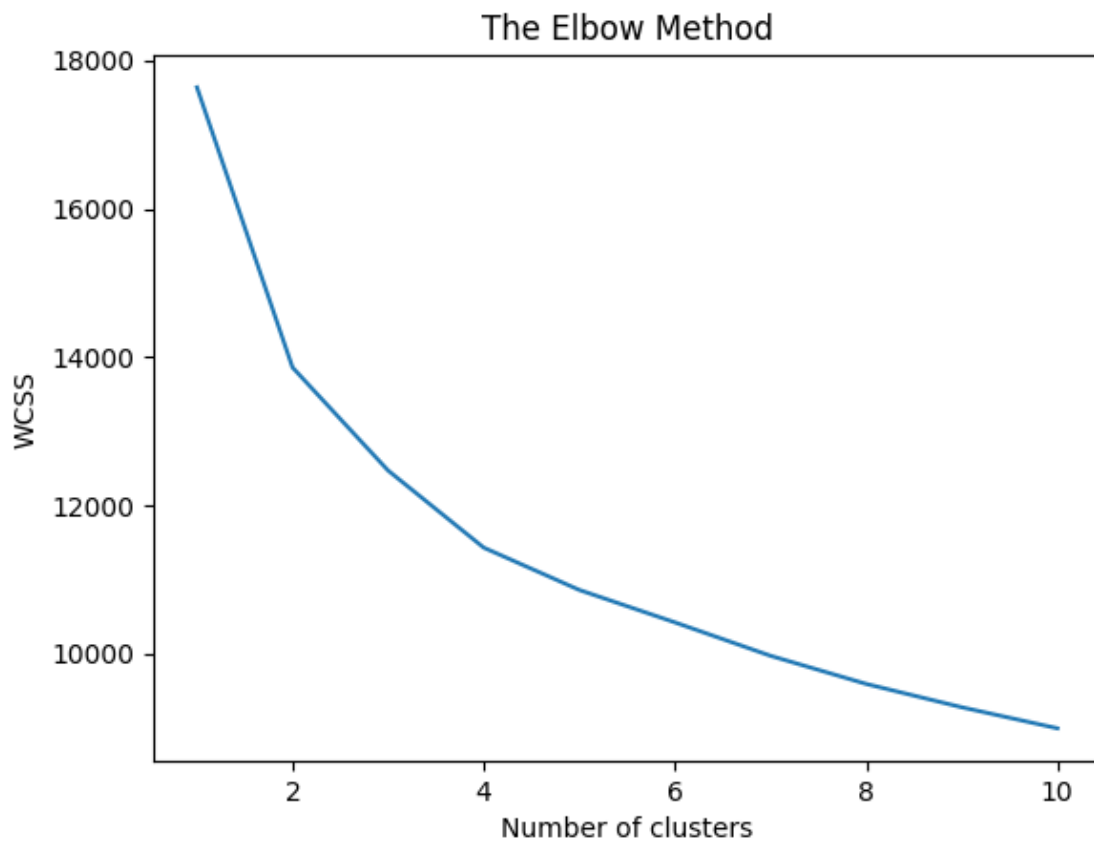
	loudness	speechiness	tempo	valence	popularity	duration_ms
0	-1.720100	1.257773	-0.214701	-2.219140	0.991720	-3.420223
1	0.675385	0.605590	0.247349	-1.139878	1.056048	0.110021
2	0.617518	1.324661	0.200931	-1.159749	1.056048	0.208697
3	0.369629	1.878737	0.298456	-1.797329	0.926593	0.600228
4	0.548482	0.981334	0.216569	-1.575715	0.926593	0.593529
...	...	...	...	...	...	...
1605	-0.834195	-0.545915	1.646976	1.740602	1.366767	-1.086609
1606	-0.891976	0.804433	-0.073846	-0.618404	1.182435	0.029702
1607	-0.472284	0.502600	-0.998899	1.108863	0.793804	-0.777394
1608	-0.929418	0.209472	-0.780756	-0.254574	0.587675	-1.612385
1609	-0.595717	-0.893421	0.038623	1.750343	1.119610	-0.607377

[1604 rows x 11 columns]

```
[20]: from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

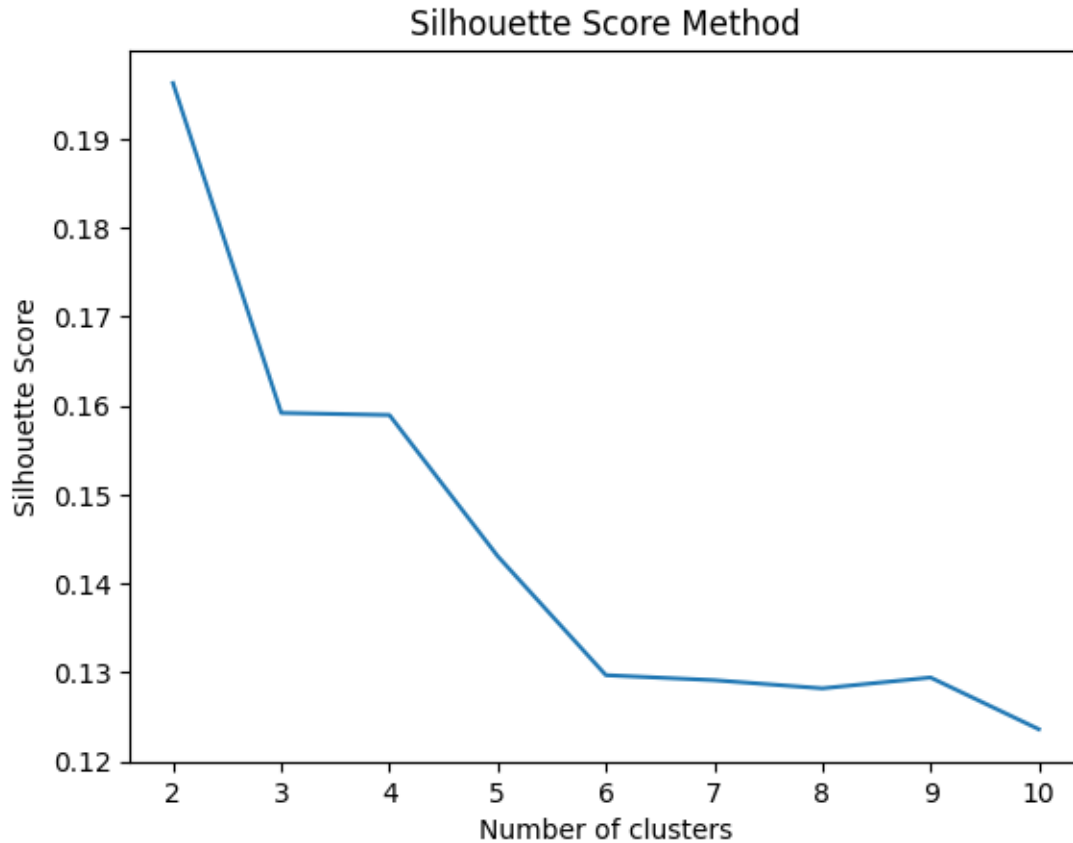
wcss = []
for i in range(1, 11):
    model = KMeans(n_clusters = i, n_init=10, init = 'k-means++', random_state=
↳42)
    model.fit(ndf_encoded_nums)
    wcss.append(model.inertia_)
plt.plot(range(1, 11), wcss)
plt.title('The Elbow Method')
plt.xlabel('Number of clusters')
```

```
plt.ylabel('WCSS')
plt.show()
```



```
[21]: # Silhouette Score method
silhouette_scores = []
for i in range(2, 11):
    model = KMeans(n_clusters=i, n_init=10, init='k-means++', random_state=42)
    model.fit(ndf_encoded_nums)
    score = silhouette_score(ndf_encoded_nums, model.labels_)
    silhouette_scores.append(score)

plt.plot(range(2, 11), silhouette_scores)
plt.title('Silhouette Score Method')
plt.xlabel('Number of clusters')
plt.ylabel('Silhouette Score')
plt.show()
```



```
[36]: # Ensure ndf is deduplicated to match the data used for metrics
ndf = ndf.drop_duplicates().copy()

categorical_cols = [
    col for col in ndf.columns
    if col.startswith("decade")]
categorical_indices = [ndf.columns.get_loc(col) for col in categorical_cols]
X_matrix = ndf.to_numpy()
```

```
[37]: !pip install kmodes
```

```
Requirement already satisfied: kmodes in /usr/local/lib/python3.12/dist-packages
(0.12.2)
Requirement already satisfied: numpy>=1.10.4 in /usr/local/lib/python3.12/dist-
packages (from kmodes) (2.0.2)
Requirement already satisfied: scikit-learn>=0.22.0 in
/usr/local/lib/python3.12/dist-packages (from kmodes) (1.6.1)
Requirement already satisfied: scipy>=0.13.3 in /usr/local/lib/python3.12/dist-
packages (from kmodes) (1.16.3)
Requirement already satisfied: joblib>=0.11 in /usr/local/lib/python3.12/dist-
```

packages (from kmodes) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in
/usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.22.0->kmodes)
(3.6.0)

```
[53]: from kmodes.kprototypes import KPrototypes
      from sklearn.metrics import silhouette_score, davies_bouldin_score, \
      ↪calinski_harabasz_score
      import matplotlib.pyplot as plt

      # Cost: lower is better, look for elbow
      # Silhouette: higher is better
      # Davies-Bouldin: lower is better
      # Calinski-Harabasz: higher is better

      costs = []
      silhouette_scores = []
      db_scores = []
      ch_scores = []

      for i in range(2, 11):
          model = KPrototypes(
              n_clusters=i,
              init="Cao",
              n_init=10,
              random_state=42
          )

          labels = model.fit_predict(
              X_matrix,
              categorical=categorical_indices
          )

          costs.append(model.cost_)

          # Use ndf_encoded_nums (numerical data) for silhouette, DB, and CH scores
          silhouette_scores.append(
              silhouette_score(ndf_encoded_nums, labels)
          )

          db_scores.append(
              davies_bouldin_score(ndf_encoded_nums, labels)
          )

          ch_scores.append(
              calinski_harabasz_score(ndf_encoded_nums, labels)
          )
```

```

)

# Plot results
plt.figure(figsize=(14, 10))

plt.subplot(2, 2, 1)
plt.plot(range(2, 11), costs, marker="o")
plt.title("The Elbow Method (Cost)")
plt.xlabel("Number of clusters")
plt.ylabel("Cost")

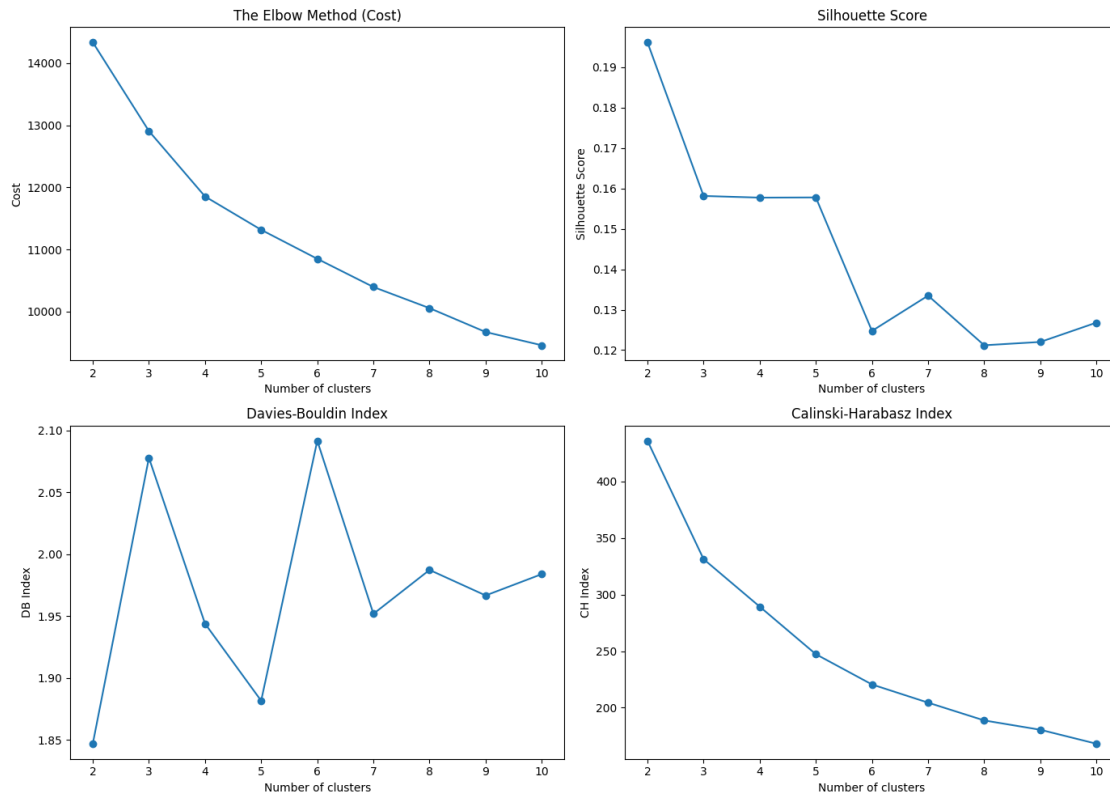
plt.subplot(2, 2, 2)
plt.plot(range(2, 11), silhouette_scores, marker="o")
plt.title("Silhouette Score")
plt.xlabel("Number of clusters")
plt.ylabel("Silhouette Score")

plt.subplot(2, 2, 3)
plt.plot(range(2, 11), db_scores, marker="o")
plt.title("Davies-Bouldin Index")
plt.xlabel("Number of clusters")
plt.ylabel("DB Index")

plt.subplot(2, 2, 4)
plt.plot(range(2, 11), ch_scores, marker="o")
plt.title("Calinski-Harabasz Index")
plt.xlabel("Number of clusters")
plt.ylabel("CH Index")

plt.tight_layout()
plt.show()

```



```
[39]: from kmodes.kprototypes import KPrototypes
```

```
model = KPrototypes(
    n_clusters=2,
    init="Cao",
    n_init=10,
    random_state=42)

labels = model.fit_predict(
    X_matrix,
    categorical=categorical_indices
)
```

```
[42]: ndf['labels'] = labels
```

```
[47]: ndf[ndf['labels'] == 0].describe()
```

```
[47]:
```

	acousticness	danceability	energy	instrumentalness	liveness \
count	739.000000	739.000000	739.000000	739.000000	739.000000
mean	-0.260995	-0.607680	0.783885	0.011010	0.732146
std	0.896496	0.786580	0.559588	1.004974	0.761074
min	-1.429655	-2.833135	-1.332573	-0.815149	-1.451507

25%	-1.131059	-1.137526	0.531547	-0.807964	0.467419
50%	-0.408952	-0.604272	0.954940	-0.591603	1.129816
75%	0.487819	-0.060074	1.205836	0.952349	1.268310
max	1.926863	1.812646	1.472873	1.808126	1.334164

	loudness	speechiness	tempo	valence	popularity \
count	739.000000	739.000000	739.000000	739.000000	739.000000
mean	0.577906	0.695157	0.330498	-0.351419	-0.121920
std	0.793954	0.825919	0.935624	0.857615	0.731196
min	-1.896645	-1.229531	-3.416460	-2.269194	-2.308976
25%	0.035240	0.056403	-0.254453	-0.965401	-0.441609
50%	0.574295	0.782294	0.405187	-0.416644	-0.094916
75%	1.045477	1.362101	0.900645	0.254927	0.297793
max	3.153099	2.104671	2.680006	1.619260	2.052126

	duration_ms	labels
count	739.000000	739.0
mean	0.315190	0.0
std	1.025320	0.0
min	-4.087214	0.0
25%	-0.231709	0.0
50%	0.197779	0.0
75%	0.772148	0.0
max	4.103868	0.0

```
[51]: ndf[ndf['labels'] == 1].describe()
```

	acousticness	danceability	energy	instrumentalness	liveness \
count	865.000000	865.000000	865.000000	865.000000	865.000000
mean	0.220659	0.514187	-0.672367	-0.012068	-0.624706
std	1.029878	0.861990	0.781623	0.996040	0.718882
min	-1.429210	-2.246364	-2.072648	-0.815149	-1.554468
25%	-0.718744	-0.174281	-1.326019	-0.812514	-1.177677
50%	0.233631	0.625354	-0.795729	-0.616017	-0.865085
75%	1.195766	1.167461	-0.070227	0.835452	-0.292177
max	2.010263	2.703416	1.205836	1.806029	1.318805

	loudness	speechiness	tempo	valence	popularity \
count	865.000000	865.000000	865.000000	865.000000	865.000000
mean	-0.500183	-0.595739	-0.277999	0.302043	0.105544
std	0.880656	0.708256	0.965768	1.016877	1.175698
min	-3.579927	-1.652319	-2.782913	-2.324433	-2.308976
25%	-1.119437	-1.136187	-0.921741	-0.382695	-0.534143
50%	-0.656605	-0.733944	-0.293596	0.463497	0.222225
75%	0.068478	-0.265264	0.299515	1.165374	0.860634
max	1.664827	1.996874	2.409841	1.774718	3.475636

	duration_ms	labels
count	865.000000	865.0
mean	-0.266664	1.0
std	0.896331	0.0
min	-4.792040	1.0
25%	-0.854552	1.0
50%	-0.267203	1.0
75%	0.326317	1.0
max	2.873373	1.0

```
[49]: # ! conda install -c conda-forge umap-learn -y
```

```
[52]: import matplotlib.pyplot as plt
import pandas as pd

# Fit K-Prototypes with your chosen number of clusters
best_k = 2 # change this based on your metrics. Also try 4

model = KPrototypes(
    n_clusters=best_k,
    init="huang",
    n_init=10,
    random_state=42
)

labels = model.fit_predict(
    X_matrix,
    categorical=categorical_indices
)

# Use numeric version of the data for visualization
# X_matrix was created from ndf *before* the 'labels' column was added.
# So, we should use ndf's columns excluding 'labels' for X_visual.
X_visual = pd.DataFrame(X_matrix, columns=ndf.drop(columns=['labels']).columns)

# Convert categorical columns to category codes for visualization only
for col in categorical_cols:
    X_visual[col] = X_visual[col].astype("category").cat.codes

X_visual = X_visual.astype(float)

# Try UMAP first, otherwise use t-SNE
try:
    import umap.umap_ as umap

    reducer = umap.UMAP(
        n_components=2,
```

```

        random_state=42
    )

    embedding = reducer.fit_transform(X_visual)
    method_name = "UMAP"

except ImportError:
    from sklearn.manifold import TSNE

    reducer = TSNE(
        n_components=2,
        perplexity=30,
        random_state=42,
        init="pca",
        learning_rate="auto")

    embedding = reducer.fit_transform(X_visual)
    method_name = "t-SNE"

# Plot
plt.figure(figsize=(9, 7))
scatter = plt.scatter(
    embedding[:, 0],
    embedding[:, 1],
    c=labels,
    cmap="tab10",
    alpha=0.75
)

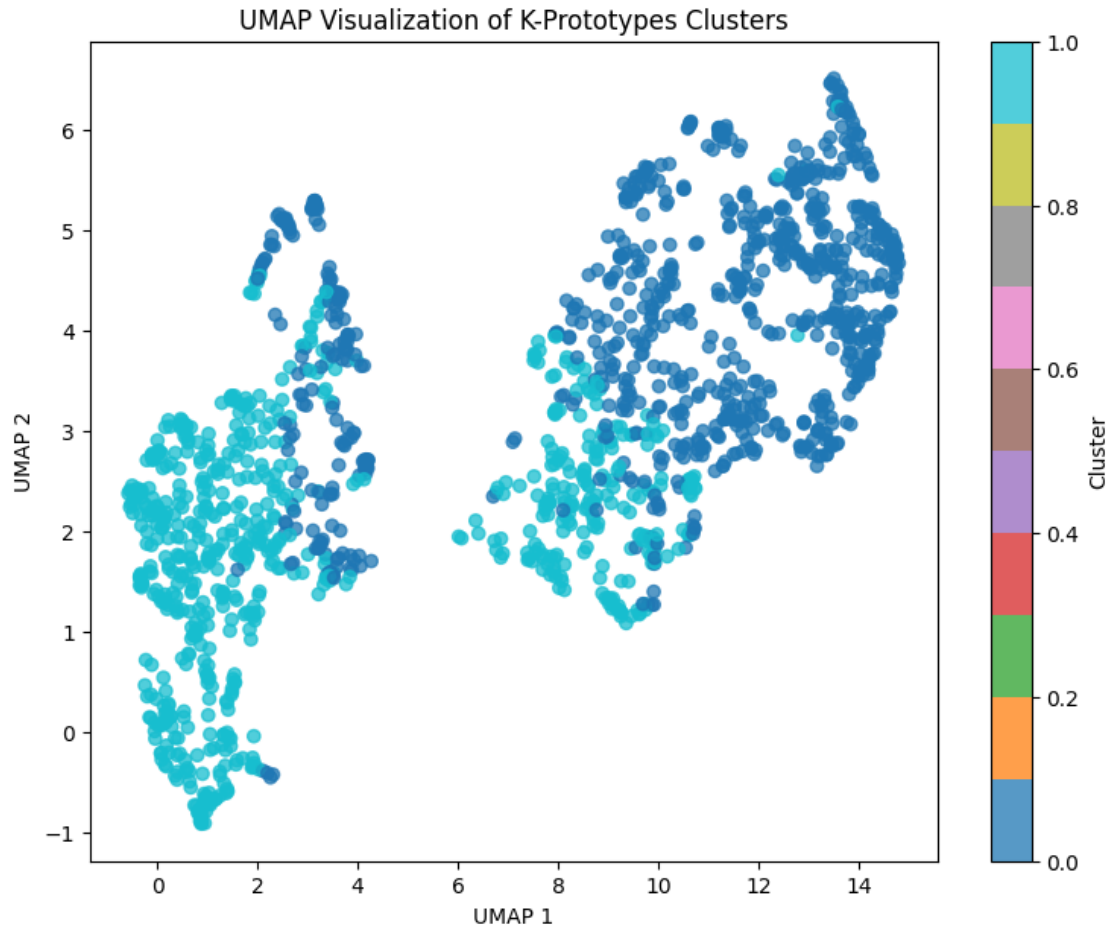
plt.title(f"{method_name} Visualization of K-Prototypes Clusters")
plt.xlabel(f"{method_name} 1")
plt.ylabel(f"{method_name} 2")
plt.colorbar(scatter, label="Cluster")
plt.show()

```

```

/usr/local/lib/python3.12/dist-packages/umap/umap_.py:1952: UserWarning: n_jobs
value 1 overridden to 1 by setting random_state. Use no seed for parallelism.
    warn(

```



Observation:

The description of cluster 1 has been successfully displayed. Additionally, the UMAP visualization for your K-Prototypes clusters has been generated and displayed, showing the distribution of your data points in a 2D space colored by their assigned cluster. You can observe the separation or grouping of the two clusters based on the features.

We have successfully performed the cluster analysis using K-Prototypes. Based on the evaluation metrics (cost, silhouette, Davies-Bouldin, and Calinski-Harabasz scores), 2 clusters were determined to be optimal. We then defined these clusters by examining their descriptive statistics and visualized them using UMAP, which showed a clear separation. This completes the cluster analysis part of the exercise.

The debugging process we undertook to successfully perform the K-Prototypes clustering and its visualization.

Here's a summary of how we resolved the issues:

Missing Libraries and Imports: We started by addressing `NameError` for `KPrototypes` and `ModuleNotFoundError` for `kmodes` by adding the necessary import statement and installing the `kmodes` library.

Data Type Consistency for Evaluation: When calculating evaluation metrics like silhouette_score, we encountered a ValueError because the metrics expected numerical data, but X_matrix contained strings. We resolved this by ensuring these metrics used ndf_encoded_nums, which is a purely numerical DataFrame.

Sample Size Discrepancies: We faced ValueError due to inconsistent sample sizes (e.g., 1604 vs. 1610 rows). This was because ndf_encoded_nums had duplicates removed, but X_matrix was based on the original ndf which still contained duplicates. We fixed this by deduplicating ndf before X_matrix was created.

Correct Label Assignment: A ValueError and KeyError occurred when assigning and accessing cluster labels. The labels were correctly generated for the deduplicated ndf, but the code was trying to apply them to the original, larger df. We corrected this by ensuring labels were assigned to ndf and referenced from ndf['labels'].

Column Mismatch for Visualization: Finally, during the visualization setup, a ValueError arose because X_matrix was created without the 'labels' column, but X_visual was trying to use ndf.columns which by then included 'labels'. We resolved this by explicitly excluding the 'labels' column from ndf.columns when creating X_visual to match X_matrix's structure.

After these adjustments, the K-Prototypes clustering was performed successfully, the clusters were defined through descriptive statistics, and the UMAP visualization was generated, showing the separation of the data points into the identified clusters.